Comunicación entre aplicaciones mediante Named Pipes

Qué es una named pipe

Una tubería es un mecanismo de comunicación entre procesos que ofrece el sistema operativo. Dos programas independientes se conectan a un canal identificado por un nombre (por ejemplo **ClinicaVetPipe**) y a través de él intercambian bytes como si fuera un fichero o un flujo de red. En .NET se usan las clases del espacio de nombres **System.IO.Pipes:**

- NamedPipeServerStream: crea la tubería y espera a que alguien se conecte.
- NamedPipeClientStream: se conecta a una tubería que ya existe usando su nombre.

La elegimos porque las dos aplicaciones se ejecutan en el mismo equipo, es más sencilla que montar sockets TCP y la comunicación puede ser en ambas direcciones

Reparto de papeles

Aplicación	Papel	Qué hace
App 2 (receptora)	**Servidor**	Crea la tubería, espera conexiones, lee el JSON, lo procesa y devuelve una respuesta
App 1 (WPF + MySQL)	**Cliente**	Consulta la base de datos, convierte los datos a JSON y los envía por la tubería

Importante: el servidor tiene que estar arrancado antes de que el cliente intente conectarse.

Flujo de una petición

1. La App 2 crea la tubería ClinicaVetPipe y se queda esperando (WaitForConnectionAsync).
2. En la App 1 el usuario pulsa un botón (por ejemplo, “Enviar mascotas”).
3. La App 1 lee los datos de MySQL y los guarda en objetos C# (Mascota, Cliente...).
4. Esos objetos se serializan a JSON con System.Text.Json.
5. La App 1 se conecta a la tubería y escribe el JSON.
6. La App 2 lee el JSON, lo deserializa a objetos y los procesa (mostrarlos, calcular algo, etc.).
7. La App 2 responde con otro JSON (por ejemplo, confirmación o resultado) y la App 1 lo muestra.

Un ejemplo del JSON puede ser:

```
{
```

```
 "tipo": "ENVIAR_MASCOTAS",
 "fechaEnvio": "2026-10-26T10:20:00",
 "datos": [
  {
   "idMascota": 1,
   "nombre": "Toby",
   "especie": "Perro",
   "raza": "Beagle",
   "fechaNacimiento": "2020-05-14",
   "peso": 12.5,
   "vacunado": true,
   "cliente": {
    "idCliente": 3,
    "nombre": "Laura",
    "apellidos": "Gómez Ruiz",
    "telefono": "612345678",
    "email": "laura.gomez@email.com"
   },
   "citas": [
    { "idCita": 10, "fecha": "2026-11-02T17:30:00", "motivo": "Vacunación anual",
"veterinario": "Dr. Pérez" },
    { "idCita": 11, "fecha": "2026-11-20T10:00:00", "motivo": "Revisión", "veterinario": "Dra.
Martín" }
   ]
  },
  {
   "idMascota": 2,
   "nombre": "Misi",
   "especie": "Gato",
   "raza": "Europeo común",
   "fechaNacimiento": "2022-03-01",
   "peso": 4.2,
   "vacunado": false,
   "cliente": {
    "idCliente": 5,
    "nombre": "Carlos",
    "apellidos": "López Sanz",
    "telefono": "698765432",
    "email": "carlos.lopez@email.com"
   },
   "citas": []
  }
 ]
}
```


En WPF, los controles (ListBox, DataGrid, TextBox...) pertenecen al hilo que los creó, el hilo de la interfaz (UI). Solo ese hilo puede modificarlos.

En la App 2 el servidor se ejecuta dentro de Task.Run, o sea, en un hilo de fondo. Si desde ahí haces directamente:

lstMascotas.ItemsSource = peticion.Datos;

salta esta excepción:

> InvalidOperationException: El subproceso que realiza la llamada no puede obtener acceso a este objeto porque el propietario es otro subproceso.

La solución: el Dispatcher
Cada ventana WPF tiene un Dispatcher, que es la cola de trabajo del hilo UI. Con Dispatcher.Invoke el hilo de fondo le pasa el código que modifica la interfaz y el hilo UI lo ejecuta cuando le toca.